CLC _____ Number _____

UDC _____ Available for reference □Yes □No

Southern University of Science and Technology

Undergraduate thesis report

Thesis Title: **ABIscope —Differential Analysis of Linux ABI Compatibility in Rust-based Kernels**

Student Name: **杨若谷**

Student ID: **12213043**

Department: **Computer Science and Engineering**

Program: **Computer Science and Technology**

Thesis Advisor: **张殷乾**

May 03, 2026

COMMITMENT OF HONESTY

1. I solemnly promise that the paper presented comes from my independent research work under my supervisor's supervision. All statistics and images are real and reliable.

2. Except for the annotated reference, the paper contents no other published work or achievement by person or group. All people making important contributions to the study of the paper have been indicated clearly in the paper.

3. I promise that I did not plagiarize other people's research achievement or forge related data in the process of designing topic and research content.

4. If there is violation of any intellectual property right, I will take legal responsibility myself.

Signature:

Date:

ABIscope —Differential Analysis of Linux ABI Compatibility in Rust-based Kernels

杨若谷

(Computer Science and Engineering Thesis Advisor: 张殷乾)

Abstract: The Linux kernel is the core component of the Linux operating system, responsible for providing essential services and managing hardware resources. With the emergence of Rust as a systems programming language, there has been growing interest in using Rust for kernel development due to its memory safety features. However, ensuring ABI (Application Binary Interface) Compatibility between Rust-based kernels and existing Linux kernel modules is crucial for the successful adoption of Rust in kernel development. Existing tools like Syzkaller target finding bugs in the Linux kernel, but they do not specifically address the challenges of ABI Compatibility in Rust-based kernels. In this thesis, we present ABIscope, a novel tool designed to generate user-space test cases for differential analysis of Linux ABI Compatibility in Rust-based kernels. ABIscope makes use of LLMs to generate test cases. We evaluate ABIscope on Asterinas, a representative Rust-based kernel, and demonstrate its effectiveness in identifying ABI Compatibility issues, providing valuable insights for developers and researchers in the field of kernel development.

[Key Words]: Linux Kernel, Rust, ABI Compatibility, LLMs, Syzkaller

[摘要]：

Linux 内核是操作系统的核心组件，负责提供系统基础服务并管理硬件资源。近年来，Rust 语言由于其内存安全和线程安全的特性，在内核开发中的应用越来越受关注。然而，在现有 Linux 生态下，采用 Rust 重写或开发的内核模块必须保证与传统 C 语言内核之间的 ABI（应用程序二进制接口）兼容性。由于内核接口涉及复杂的系统调用和底层数据结构，即便没有发生崩溃，也可能产生细微的行为差异。现有的内核模糊测试工具（例如 Syzkaller）主要面向崩溃挖掘和漏洞检测，缺乏对跨内核 ABI 兼容性的直接支持。

针对上述问题，本文提出了 ABIscope——一种基于大语言模型（LLM）驱动的用户态测试用例生成与差分分析框架。我们提出了一种包含种子初始化、大模型结构与参数变异、差分执行验证以及覆盖率引导选择的闭环工作流。具体而言，本方法不依赖于内核内嵌插桩，而是设计了系统调用参数覆盖率（SAC）机制，从变异代码的静态特征和运行时的副作用等多维度进行反馈引导。

通过对比测试标准 C 语言内核与基于 Rust 构建的新型内核，ABIscope 能够自动化生成涵盖大量边界条件（Edge Cases）的系统调用序列，并结构化地分析两者在返回值、错误码及文件系统副作用上的差异。实验结果表明，ABIscope 能够有效定位并归因 ABI 不兼容问题，为未来基于 Rust 的系统内核开发和生态平滑过渡提供了有力工具和方法支持。

[关键词]： Linux 内核；Rust；ABI 兼容性；大语言模型；Syzkaller

目录

1. Introduction **1**

2. Background and Related Work **4**

2.1 Linux kernel and Rust-based kernels 4

2.2 ABI Compatibility 5

2.3 Fuzzing and Testing Techniques 6

2.4 Large Language Models in Software Testing 8

3. Methodology **9**

3.1 Overview 9

3.2 Seed Initialization 10

3.2.1 Syzkaller and Syzlang Format 10

3.2.2 LLM-based Syzlang-to-C Translation with Probe Injection and Self-Repair 11

3.3 LLM-based mutation 11

3.4 Execution and Validation 12

3.5 Coverage-Guided Selection 13

3.5.1 Formal Definition of SAC 13

3.6 Threats to Validity 15

4. Experimental Results **16**

4.1 Experimental Setup 17

4.2 Overall Results 17

4.3 Filesystem Syscall Compatibility 17

4.3.1 Errno Mapping Divergences 18

4.3.2 Missing `openat2` Support 18

4.3.3 File-Descriptor Lifecycle Differences 18

4.4 I/O Syscall Compatibility 19

4.4.1 `dup2` Returns `ENOENT` Instead of `EBADF` 19

4.4.2 `fcntl(F_GETFL)` Missing `O_LARGEFILE` Bit 19

4.4.3 Access-Mode Check Divergence 20
4.4.4 Umask and File-Creation Mode Difference 20
4.4.5 `fdatasync` Stub Behavior 20
4.5 Memory Management Syscall Compatibility 20
4.5.1 `mmap` Validation Order (68 Cases) 21
4.5.2 `madvise` Incomplete Implementation 21
4.5.3 Struct ABI and `copy_to_user` Mismatches 21
4.6 SAC Coverage Analysis 21
5. Summary and Future Work 22
5.1 Summary 22
5.1.1 Contributions and Research Significance 23
5.1.2 Limitations 23
5.2 Future Work 24
References 26

1. Introduction

Linux operating system has been widely adopted in various domains from personal computers to servers and embedded systems[1]. The Linux kernel is the core component of the Linux operating system. It is essential for providing different kinds of crucial services like process management, file system and so on. Linux kernel is written mostly in C, which is a powerful but unsafe programming language because of its lack of memory safety features. These days, a memory safety issue in kernels can finally result in a security vulnerability which can be attacked by malicious users and result in a severe consequence[2]. In 2022, an engineer discovered the Dirty Pipe vulnerability (CVE-2022-0847)[3]. This vulnerability can help an attacker get root privileges on a system if the attacker has read access to a file. Rust is a modern programming language that provides memory safety guarantees without sacrificing performance. It has become more and more popular in the systems programming community. Researchers begin to use Rust to write kernel modules and even entire kernels like KeepOS, Asterinas, Kerla and so on[4].

However, many existing Linux kernel modules like device drivers and file systems are written in C. They can not directly work with Rust-based kernels. Based on this, ideas of using Rust to write Linux kernel modules have been proposed[5]. Also, there are a large number of C modules such as device drivers and it is hard to port all of them to Rust kernels. However, ensuring ABI(Application Binary Interface) Compatibility between Rust-based kernels and existing Linux kernel modules is crucial for the successful adoption of Rust in kernel development. ABI Compatibility means that the binary interface between the kernel and user-space applications or kernel modules remains consistent, allowing them to interact seamlessly without requiring modifications[6]. Ensuring ABI Compatibility is essential for maintaining the stability and functionality of existing software while enabling the benefits of Rust's memory safety features in kernel development.

Although kernel developers have made efforts to ensure ABI Compatibility when building Rust-based kernels, there are still challenges and potential issues that can arise. For most rust-

based kernels, developers just write in the documentation that the kernel they develop is ABI compatible with the Linux kernel, but they do not provide a systematic way to verify the ABI Compatibility[7]. Existing tools like Syzkaller[8] target finding bugs in the Linux kernel, but these tools cannot be directly used in the Rust-based kernels because of the differences in the kernel implementation and the lack of support for Rust-based kernels. Other widely used testing frameworks share the same limitation. Trinity[9], a system call fuzzer, generates semi-random arguments to stress-test syscall entry points, but it is tightly coupled to Linux-specific syscall tables and assumes a Linux kernel build environment, making it unsuitable for Rust-based kernels out of the box. Similarly, the Linux Test Project (LTP)[10] provides thousands of conformance and regression tests for the Linux kernel, yet its test cases are written against Linux-specific behaviors and build infrastructure, and likewise cannot be applied to Rust-based kernels without substantial adaptation. What's more, these tools target finding bugs in the kernel, but finding bugs is not the same as finding ABI Compatibility issues. ABI Compatibility issues can be more subtle and may not be regarded as bugs in the kernel. In this case, there is a need for a systematic way to verify the ABI Compatibility of Rust-based kernels with existing Linux kernel modules.

Recently, large language models (LLMs) have shown great potential in various domains, including software development, code generation, and testing[11,12]. Many LLM-based test generation methods have been proposed, demonstrating strong capabilities in generating high-quality inputs and exploring complex program behaviors. However, directly applying these techniques to kernel-level ABI compatibility testing remains challenging due to the complexity of kernel interfaces and the lack of well-defined specifications[13]. Hallucination is also a common issue in LLM-based generation, where the model may produce programs that seem correct but are actually invalid or irrelevant to the target domain. This can lead to a large number of invalid test cases which may cause unnecessary computation overhead and reduce the effectiveness of testing.

In particular, the Linux ABI is highly complex, involving numerous system calls, complicated data structures, and subtle interactions between user space and kernel space. Exhaustively verifying ABI compatibility across all possible interfaces is therefore infeasible in practice. To address this challenge, we propose to focus on a critical and well-defined subset of the ABI: system calls.

System calls are the only legitimate boundary through which user-space programs can interact with the kernel. All other observable kernel behaviors — including file I/O, process management, inter-process communication, memory mapping, and device access — are ultimately funneled through this interface. Higher-level constructs such as the C standard library, language runtimes, and even container or virtualization layers eventually bottom out in system calls. As a result, if a Rust-based kernel faithfully reproduces the syscall ABI of Linux, the vast majority of unmodified user-space binaries should run correctly on top of it. Conversely, any divergence at this layer will surface as visible misbehavior in real workloads. This makes system calls not only a well-scoped and tractable target, but also the single most informative layer at which to probe ABI compatibility.

However, system calls cannot be treated as independent, stateless functions. They are deeply stateful, and most meaningful syscalls only make sense within a sequence that establishes the required kernel state. A typical file-access workflow, for instance, follows the chain **open-write-read-close**: the file descriptor returned by `open` is a prerequisite for `write` and `read`, and `close` is only meaningful given a previously opened descriptor. A naive testing strategy that generates syscalls and arguments uniformly at random will have uncertain probability, invoke operations on invalid descriptors, unmapped addresses, or non-existent objects. Such tests provide little signal about ABI compatibility, because they never reach the parts of the kernel where divergence between Linux and a Rust-based reimplementation is most likely to manifest. An effective ABI compatibility tester must therefore be aware of these dependency chains and generate syscall sequences that respect resource lifetimes, rather than treating each call in isolation.

Based on this insight, we design and implement ABIscope, a seed generation tool tailored for ABI compatibility testing of Rust-based kernels. ABIscope leverages LLMs to generate system call sequences (seeds) and incorporates an intermediate parameter propagation mechanism, where arguments from previous system calls are reused and transformed for subsequent calls. This design enables ABIscope to construct semantically meaningful and stateful test cases, rather than isolated and random inputs.

Furthermore, ABIscope improves system call argument coverage by systematically exploring diverse parameter combinations and dependencies. By increasing the diversity and validity of system call inputs, ABIscope indirectly enhances kernel code coverage, as measured by KCOV. This approach allows us to effectively exercise deeper execution paths in the kernel without requiring intrusive instrumentation or kernel-specific adaptations.

We evaluate ABIscope on the file system, I/O, and memory management subsystems of Asterinas, a representative Linux ABI-compatible Rust-based kernel, and compare its behaviors against the Linux kernel. Our results reveal several previously undocumented ABI incompatibility issues, demonstrating that even kernels claiming ABI compatibility may exhibit inconsistencies in system call semantics and behaviors. These findings highlight the necessity of systematic ABI verification and showcase the effectiveness of ABIscope in uncovering such issues.

2. Background and Related Work

In this section, we will review the background and related work in the areas of Linux kernel development, Rust-based kernels, ABI compatibility, fuzzing and testing techniques and also the application of large language models in the software testing field.

2.1 Linux kernel and Rust-based kernels

Linux kernel is developed based on Unix-like kernel and it is written mostly in C. The key design of the Linux kernel is monolithic kernel[14], which means that all core os services runs in a single kernel address space and interact via direct function calls. In this design, there will be no IPC, no message passing, no internal context switches and so on. IPC and context

switch in microkernel design will cost microseconds level latency, but in monolithic kernel design, the latency of the function calls can be reduced to nanoseconds level. In this case, linux kernel can provide better performance than the microkernel design and this design helps the Linux kernel easy to integrate with new hardware and software. However, the monolithic kernel design also has some drawbacks. because all components share the kernel space, a bug in one component can potentially crash the entire system. Due to the complex interactions between different components, it can be difficult to isolate and debug issues. The c programming language itself is a powerful but unsafe programming language because of its lack of memory safety features. This increases the risk of security vulnerabilities and system crashes[15]. Rust, a modern systems programming language, has been gaining popularity in recent years due to its memory safety, concurrency safety and high performance without relying on a garbage collector[16]. It achieves this through a strong static type system and its ownership and borrowing model. Rust can prevent common memory bugs at compile time including buffer overflows, use-after-free and so on. It eliminates a large class of vulnerabilities common in C/C++[17]. Because of these advantages, there has been growing interest in using Rust for kernel development. The Linux kernel community has started to accept Rust code and there are already some Rust-based kernel like Asterinas[4], rCore[18] and Kerla[19]. However, despite these benefits, an important open question remains: whether Rust-based kernels behave consistently with the Linux kernel at the Application Binary Interface (ABI) level. Such consistency is essential for ensuring compatibility and correctness across different kernel implementations.

2.2 ABI Compatibility

ABI (Application Binary Interface) defines the low-level interface between an application and the operating system including system call conventions, data types and observable behaviors such as return values, side effects and error codes. ABI compatibility means that a program compiled staticly for a given ABI can run correctly on another kernel implementing the same ABI without modification. Also, they are expected to produce consistent observable behaviors like the same return values or the same return error codes when executing the same system calls

with the same arguments[6]. In the context of kernel development, many applications and kernel modules are built on top of the Linux kernel ABI. Many programs rely on specific behaviors of the Linux kernel, some of the programs may even rely on specific return values or error codes. Many library developers use `switch` or other control flow statements to handle different return values and error codes. That is to say, even if the Rust-based kernels can successfully execute the same system calls, they may still produce different return values or error codes which may result in unexpected behaviors of the applications if they do not ensure ABI Compatibility with the Linux kernel. In this case, researchers find it crucial to evaluate the ABI Compatibility of Rust-based kernels with the Linux kernel. However, evaluating ABI Compatibility is a non-trivial task. Many kernels which claims that they are Linux-compatible seldom give a detailed analysis of their ABI compatibility with the Linux kernel. A static analysis tool called System Call Matching Language(SCML) is proposed to analyze the syscall interface of a kernel and compare it with the Linux kernel. However, it only focuses on the flags and bits that implemented in the kernel and it does not evaluate the return values, error codes and side effects of the system calls. To the best of our knowledge, there is no existing framework or benchmark that can systematically evaluate the ABI Compatibility of Rust-based kernels with the Linux kernel.

2.3 Fuzzing and Testing Techniques

Fuzzing is a widely used technique for finding bugs and vulnerabilities in software. It involves generating random or semi-random inputs and feed them to a specific target program to trigger unexpected behaviors or crashes[20]. These kinds of programs are called fuzz drivers or fuzz targets. Fuzz drivers can be generated manually or automatically. They ensures a sequence of API calls with specific arguments to trigger specific code paths in the target program. Because developing fuzz drivers manually can be time-consuming, there are many existing researches on automatic fuzz driver generation[21,22]. Other researches focus on the input generating process and they use machine learning techniques to generate more effective inputs[23]. However, most of the existing fuzzing techniques are designed for testing user-space applications and they are not directly applicable to kernel testing. Kernel fuzzing is more challenge than user-space

fuzzing because of the complexity of the kernel and the difficulty of reproducing bugs. There are some existing kernel fuzzing tools like Syzkaller[8],Trinity[9] and so on. However, these tools are designed mainly for finding bugs and vulnerabilities in the Linux kernel and it may take great effort to port them to do kernel fuzzing on other kernels. Also, fuzzing techniques are not designed for evaluating ABI Compatibility because they do not focus on the return values, error codes and side effects of the system calls. They find bugs by triggering crashes or hangs, but they do not evaluate the correctness of the behaviors of the system calls.

Except fuzzing, there are also some other testing techniques for kernel testing like unit testing, integration testing and so on. Unit testing is a software testing technique that focuses on testing individual units or components of a software system in isolation. It can help to find bugs that will not cause crashes but can cause unexpected behaviors[24]. Integration testing is a software testing technique that focuses on testing the interactions between different components of a software system. It can help to find bugs that arise from the interactions between different components[25]. However, these testing techniques are not designed for evaluating ABI Compatibility because they do not focus on the return values, error codes and side effects of the system calls. They are designed for testing the functionality of the software rather than evaluating the compatibility of the software with other implementations. Differential testing is a software testing technique that focuses on testing the same program in different environments or with different implementations. This can help us to find incompatibility issues between different kernels because if two kernels claim that they pursue the same ABI, they are expected to produce the same return values, error codes and side effects when executing the same system calls with the same arguments. Through this kind of testing, we can find the incompatibility issues between the Rust-based kernels and the Linux kernel. However, there is no existing framework or benchmark that can systematically evaluate the ABI Compatibility of Rust-based kernels with the Linux kernel through differential testing.

2.4 Large Language Models in Software Testing

LLM(Large Language Models) is a type of machine learning model that is train on a large corpus of dataset based on the transformer architecture[26]. A well-known example of LLM is GPT developed by OpenAI[27]. The core idea of LLM is to learn the statistical patterns and relationships in the training data and use the knowledge to generate new date or make predictions based on the given sequence of previous tokens. LLM has been applied in various fields including natural language processing, computer vision and so on. Compared to the traditional machine learning models, one of the great advantages of LLM is that LLM has a context-awared ability which means that LLM can understand the context of the input in nature language and generate more relevant and coherent output. This is extremely outstanding in code generation tasks. LLM can generate code snippets that strictly follow the instructions and requirements given in the prompt, while other machine learning models may not be able to generate code that is syntactically correct or semantically meaningful when following the prompts[28].

In the software testing field, LLM has two main ways to be applied: static and dynamic. In static applications, researchers can directly feed the source code of the target program to the LLM and ask it to analyze the code to see whether there are any potential bugs or vulnerabilities in the code[29]. This kind of application is called static code analysis. It is quite convenient because it does not require the target program to be executed and it can analyze the code in a more comprehensive way. However, this kind of method suffer heavily in false negative because the LLM may not be able to understand the code completely if the code is too complex or the code comes from a domain that the LLM is not familiar with. Another way to apply LLM in software testing is dynamic application which means that we can use LLM to generate tests cases for the target program and then execute the test cases to see whether there are any bugs or vulnerabilities in the target program. This kind of application is called dynamic test case generation. It some specific area, it performs better than the static code analysis because it can generate test cases that can trigger specific code paths in the target program and it can also evaluate the behaviors of the target program when executing the test cases, while static analysis

suffer from complex code and perform badly. However, dynamic test case generation also has some drawbacks. It may require a lot of compuational resources to generate the test cases. Also, the tests cases can sometimes be too strict or too simple. But when it comes to evaluating ABI Compatibility, dynamic test case generation is a more powerful method because we need to see the detailed output of different kernels. Therefore, leveraging LLMs to automatically generate dynamic test cases presents a promising approach to systematically evaluate and uncover ABI inconsistencies between Rust-based kernels and the Linux kernel.

3. Methodology

3.1 Overview

We present ABIscope, a novel tool designed to generate user-space test cases for differential analysis of Linux ABI Compatibility in Rust-based kernels targeting syscalls. The overall pipeline follows an iterative seed generation and evaluation process, leveraging LLMs with feedback-driven exploration. At a high level, ABIscope consists of four key stages: (1) Seed initialization (Section 3.2), (2) LLM-based mutation (Section 3.3), (3) execution and validation (Section 3.4), and (4) coverage-guided selection (Section 3.5). These four key components form a closed feedback loop. The overall pipeline is illustrated in Figure 1. We describe each stage in detail in the following sections.

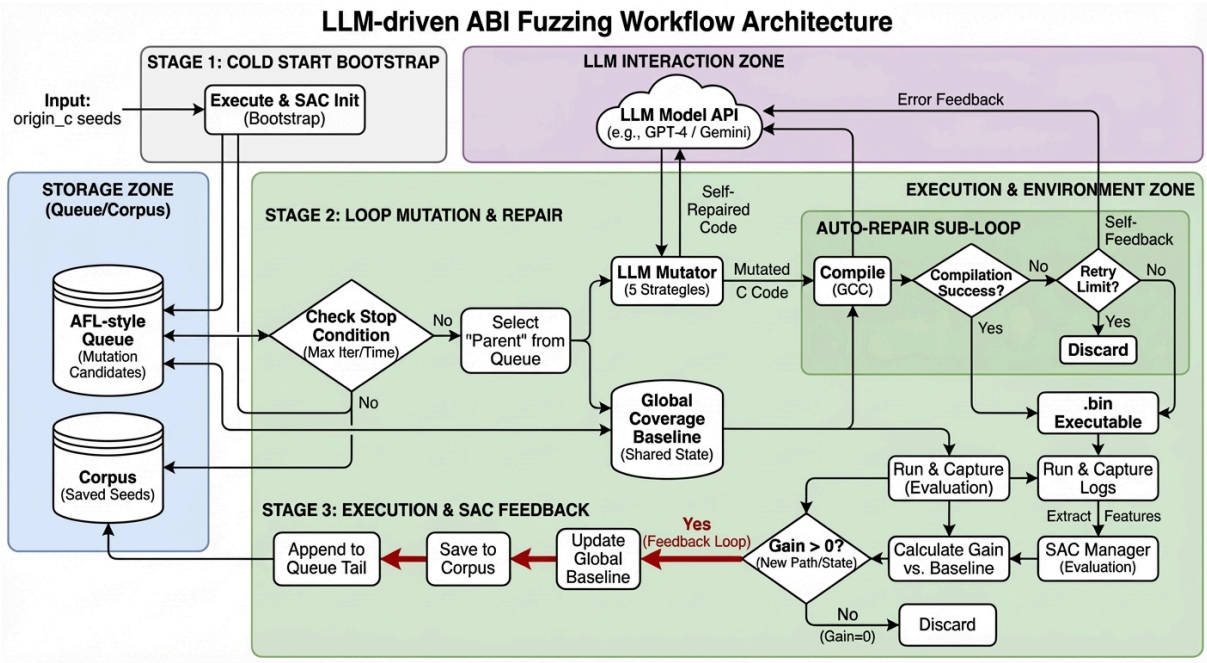

Figure 1 Overall pipeline of ABIscope.

3.2 Seed Initialization

Linux ABI Compatibility is complicated by the fact that the Linux kernel provides a large number of system calls and different structures, each of them have their own ABI. It is impractical to generate test cases for all of them. To simplify the problem, we focus on a subset of system calls that are commonly used and have well-defined ABIs. We select a set of system calls of file system, I/O and memory management which are commonly used in kernel development and important for kernel stability and performance. We will generate programs in these three categories of system calls as the initial seeds for the mutation process.

3.2.1 Syzkaller and Syzlang Format

Syzkaller is a popular fuzzing tool for finding bugs in the Linux kernel developed by Google. It generates random inputs for system calls and monitors the kernel's behavior to identify potential vulnerabilities. But we cannot directly use Syzkaller for ABI Compatibility testing because incompatibility issues may not necessarily cause crashes or other easily detectable symptoms[30]. Although the target of Syzkaller is to find bugs in the target kernel, it can still generate valid programs that involve syscalls we are interested in. Syzkaller seeds are written in Syzlang, a strong domain specific language designed for syscall sequences with

explicit argument types, virtual memory address, typed file handles and so on. Syzlang is able to represent the structural intent of a test like syscalls to call and its order. However, Syzlang itself is not directly compilable, so we need to convert the Syzlang seeds generated by Syzkaller into C programs.

3.2.2 LLM-based Syzlang-to-C Translation with Probe Injection and Self-Repair

Syzkaller itself provides a tool to convert Syzlang seeds into C programs, but the result C programs provided by the tool are not always valid and compilable. Even if a program can compile and execute, it cannot guarantee the sandbox environment of the ABI testing process thus not suitable for our purpose. Also, our target is to see the return values and error codes of the syscalls, but the raw programs do not provide the mechanism to collect these information. To address these issues, we design a LLM-based Syzlang-to-C translation process with probe injection. The probe macro injection is a static rewriting process that scans the C source code for syscall invocations and inserts two types of macros: `LOG_SYSCALL_RES(name,ret)` and `LOG_STAT(name,ret,st)`. The former is used to record syscall name, return value and errno, while the latter is used for stat-family calls to record file attributes such as size, mode and so on. These probes will be executed at runtime to collect the necessary information for the future analysis. If compilation or execution fails, the error messages are fed back to the LLM to fix the test cases(up to 3 times in default). Programs that still fail after all repair attempts will be discarded. The valid and runnable C programs generated in this stage will be used as the initial seeds for the mutation process.

3.3 LLM-based mutation

ABIscope adopts a mutation-based approach to generate new test cases from the initial seeds. We leverage LLMs to perform mutations on the valid test cases generated in the previous stage. The LLM takes the valid test cases as input and generates new test cases by applying various mutation strategies.

- Structure mutation: LLM can insert new syscalls to the test cases which can trigger new code paths in the kernel. For example, LLM can insert `mmap` syscall before `read` syscall to trigger

the code path of memory mapping in the kernel. Also, LLM can remove some syscalls from the test cases to reach the edge cases. For example, LLM can remove `open` syscall from the test case to trigger the code path of handling invalid file descriptors to see the ability of the error-handling code in the rust-based kernel.

- Order mutation: LLM can change the order of the syscalls in the test cases to trigger different code paths in the kernel. For example, LLM can change the order of `rmdir` and `rename` syscalls to trigger the code path of handling directory operations in the kernel.
- Fault injection: LLM can inject faults into the test cases to trigger specific error-handling code paths in the kernel. We can find incompatibility issues through the different errno values returned by the syscalls.
- Logic mutation: LLM can construct stateful misuse patterns like double free, use-after-free and so on. The idea is the same as the logic mutation in AFL++ to see the compatibility of error-handling.
- Parameter mutation: LLM perturbs syscall arguments at both value and structure levels to expose ABI-sensitive boundary behavior. Concretely, it rewrites scalar parameters (e.g., file descriptors, offsets, lengths, and flags) toward the cases that never appear in the seeds before. For instance, it may generate negative offsets, oversized lengths, or uncommon flag combinations. LLM can effectively explore the argument space by learning from the existing seeds.

3.4 Execution and Validation

After generating new test cases through LLM-based mutation, ABIscope executes these test cases on Linux first. If the test case can be executed successfully on Linux, we will execute the same test case on Rust-based kernels. If the generated programs cannot be executed successfully on Linux, we will involve another LLM to fix the test cases. If the repair process fails after limited attempts, we will discard the test cases. The execution results of the test cases on both Linux and Rust-based kernels will be collected and compared to evaluate the compatibility of the test cases. We will compare the return values, output parameters, and error

codes of the syscalls to identify any discrepancies that may indicate ABI incompatibility issues. If we find any discrepancies, we will further analyze the test cases to understand the root cause of the incompatibility and write reports to summarize the issues we found.

3.5 Coverage-Guided Selection

To quantify the effectiveness of LLM-generated mutations, ABIscope adopts SAC (System-call Argument Coverage) as its primary feedback signal. In contrast to conventional control-flow coverage (e.g., line, basic-block, or branch coverage collected through kernel instrumentation), SAC evaluates whether a mutated seed explores previously unseen argument semantics and externally observable execution outcomes. This formulation is particularly suitable for ABI compatibility analysis, where behavioral divergence may arise even when no crash or obvious control-flow anomaly is observed.

In our implementation, SAC integrates both static and dynamic evidence. The static dimension is extracted directly from generated C programs and includes: (i) flag-level behaviors, where individual macros (such as `O_RDWR`, `O_CREAT`, and `SEEK_END`) and their co-occurrence relations are encoded; (ii) path-pattern abstractions, where path-like literals are mapped to semantic categories defined by preconfigured pattern rules; (iii) numeric and alignment-sensitive arguments, including boundary values (e.g., `-1`, `0`), page-aligned constants, and oversized unsigned probes; and (iv) permission-mask semantics, where mode-related expressions are matched against predefined privilege-related masks to detect bits such as SUID/SGID.

3.5.1 Formal Definition of SAC

Let $\sigma = (c, \ell)$ denote a seed, where c is the C source program and ℓ is the runtime probe log obtained after compilation and execution. The SAC feature extraction function Φ maps each seed to a set of symbolic feature tokens:

$$\Phi(\sigma) = \Phi^s(c) \cup \Phi^d(c, \ell) \tag{3.5.1.1}$$

where Φ^s denotes the static feature extractor operating on the source text, and Φ^d denotes the dynamic feature extractor operating on the runtime log.

The static features are decomposed into five disjoint categories:

$$\Phi^s(c) = \Phi^{\text{flag}}(c) \cup \Phi^{\text{path}}(c) \cup \Phi^{\text{int}}(c) \cup \Phi^{\text{mode}}(c) \cup \Phi^{\text{bound}}(c) \tag{3.5.1.2}$$

Specifically:

- $\Phi^{\text{flag}}(c)$ captures individual flag tokens $f \in \mathcal{F}$ and their line-level co-occurrence combinations. Each f contributes token `FLAG:`f; any pair (f_1, f_2) co-occurring on the same source line contributes `PAIR:`$f_1|f_2$; the full co-occurring set contributes a `COMBO:` token.
- $\Phi^{\text{path}}(c)$ maps path-like string literals to semantic category tags (e.g., `PATH_DOT`, `PATH_DEEP_NEST`) via a set of predefined regular-expression rules.
- $\Phi^{\text{int}}(c)$ records numeric boundary tokens including 0, -1, page-aligned constants $(4096, 8192)$, $2^{31} - 1$, the sentinel -100 (`AT_FDCWD`), and overflow-range probes ($\geq$ 0xFFFFFFFF).
- $\Phi^{\text{mode}}(c)$ detects permission-mask bit patterns (SUID $=$ 0o4000, SGID $=$ 0o2000, sticky bit $=$ 0o1000, etc.) within mode-parameter argument positions only, avoiding false positives from unrelated numeric literals.
- $\Phi^{\text{bound}}(c)$ generates call-site-bound tokens of the form STATIC:s:α for each syscall $s \in \mathcal{S}$ and each attribute α derived from its argument list, binding argument semantics to the specific call site rather than accumulating them globally.

Dynamic features are extracted per syscall occurrence. For the k-th occurrence of syscall n in log ℓ, denoted $n^{(k)}$, the extractor produces:

$$\Phi^d(c, \ell) = \bigcup_{n^{(k)} \in \ell} \{\text{DYN:SYSCALL:}n, \text{DYN:SYSCALL_OCC:}n\#k\} \cup O\big(n^{(k)}\big) \cup E\big(n^{(k)}\big) \cup S\big(n^{(k)}\big) \tag{3.5.1.3}$$

where $O\big(n^{(k)}\big)$ records the success/failure outcome and the exact return value; $E\big(n^{(k)}\big)$ records the concrete errno value on failure; and $S\big(n^{(k)}\big)$ records filesystem side-effects—file-size regime, link-count transition, and effective permission-mode bits—observed via `stat`-family probes. For syscalls that produce no observable filesystem side-effects (e.g., memory-management or pure I/O operations), $S\big(n^{(k)}\big) = \emptyset$.

Let G_t denote the global coverage set accumulated from the first t accepted seeds. For a candidate seed σ, the **coverage gain** is defined as the cardinality of the set difference between the seed's feature set and the current global coverage:

$$\text{gain}(\sigma, G_t) = |\Phi(\sigma) \setminus G_t| \tag{3.5.1.4}$$

A seed is admitted into the corpus if and only if it introduces at least one novel feature token:

$$\sigma \text{ is accepted if and only if } \text{gain}(\sigma, G_t) > 0 \tag{3.5.1.5}$$

Upon acceptance, the global coverage set is updated monotonically:

$$G_{t+1} = G_t \cup \Phi(\sigma) \tag{3.5.1.6}$$

Seeds that do not increase G_t are discarded, ensuring that the corpus grows only when genuinely new argument-semantic or execution-outcome evidence is encountered. Consequently, SAC serves as a non-instrumentation, semantics-oriented guidance mechanism that continuously steers the LLM toward high-value edge cases for ABI incompatibility discovery.

At each mutation step, ABIscope selects uniformly at random among the five mutation strategies: Structure mutation, Order mutation, Fault injection, Logic mutation, and Parameter mutation. Each is assigned equal probability $p = 1/5$. The overall pipeline repeats until the maximum number of iterations or the time budget is exhausted.

3.6 Threats to Validity

Despite the feedback-driven design of ABIscope, several validity threats still remain.

First, there are construct-validity threats in the coverage signal. SAC is designed to capture argument semantics and observable side effects, but it is still a proxy metric rather than a direct measure of incompatibility severity. A seed may produce positive SAC gain without exposing a meaningful ABI defect, while some true incompatibilities may exhibit limited gain if their behavioral signatures overlap with already observed patterns. To reduce this risk, SAC is evaluated jointly with differential execution outcomes and error-code distributions, rather than used as a standalone criterion.

Second, there are internal-validity threats in mutation and execution. LLM-based mutation is inherently stochastic and can be sensitive to prompt context, which may affect reproducibility across runs. In addition, generated tests can contain invalid or weakly constrained arguments;

such cases may amplify input noise and obscure real kernel-level divergence. We therefore retain only compilable/runnable candidates and interpret failures together with runtime traces, so that parameter invalidity can be separated from compatibility-relevant behavioral differences.

Third, differential execution itself introduces confounding factors. Linux and Rust-based kernels may differ in scheduling behavior, timing, default configuration, and tolerance to undefined or implementation-dependent behavior. Some output discrepancies can therefore reflect environmental or semantic ambiguity rather than strict ABI incompatibility. To mitigate this issue, ABIscope compares aligned test inputs under controlled conditions and emphasizes stable, repeatable divergences.

Finally, there are external-validity limitations. The current setup focuses on a selected subset of file system, I/O, and memory management syscalls and may not fully generalize to other subsystems (e.g., networking, IPC, device drivers) or production workloads. Consequently, conclusions should be interpreted as evidence for the three studied syscall domains, with broader generalization requiring expanded syscall coverage and larger-scale cross-kernel evaluation.

4. Experimental Results

The most direct way to evaluate a testing framework is to apply it to real-world software and to see whether it can find real bugs[31]. In the idea of ABIscope, the most direct way is to use the programs provided by the ABIscope and run on the both kernels. We therefore conduct end-to-end experiments that full ABIscope pipeline runs autonomously with 200 initial seeds until either a clock budget of 6 hours is exhausted or a total of 500 executable binaries are generated. All experiments are performed on a server equipped with two AMD EPYC 9354 32-core processors (128 hardware threads in total) and 1.5 TiB of RAM. LLM-based mutation and translation are powered by GPT-5-mini (version 2025-08-07) via the OpenAI API. The reference environment is based on Ubuntu 24.04 LTS with Linux kernel 6.8.0-94-generic and the target Rust-based kernel is Asterinas[4], a Linux ABI-compatible framekernel OS implemented

entirely in safe Rust that supports over 210 Linux system calls and has been shown to deliver performance comparable to the Linux kernel.

4.1 Experimental Setup

We evaluate ABIscope across three syscall domains: (1)**Filesystem**(FS) opertions including `open`, `openat`, `read`, `write`, `fcntl` and related calls; (2) **I/O Management** including `dup`, `lseek`, `ftruncate`, `fsync`, `pread64` and related calls; (3) **Memory Management** including `mmap`, `madvise`, `brk` and related calls. In all experiments, the same set of static compiled C binaries is executed on both a reference Linux kernel and the target Rust-based kernel. Runtime probe logs (`LOG_SYSCALL_RES` and `LOG_STAT` outputs) are collected for each execution and compared to identify the incompatibility issues. A case is classified a incompatiable if any return value, error code, or observable side-effect differs between the two platforms.

4.2 Overall Results

Domain	Total Cases	Incompatible	Rate	SAC Feature Tokens
Filesystem (FS)	484	420	86.8%	4,383
I/O Management (IO)	500	82	16.4%	11,161
Memory Mgmt (Mem)	478	304	63.6%	3,226

Across all three domains, ABIscope successfully generated and evaluated a total of 1,462 test cases, among which 806 were identified as exposing at least one ABI incompatibility issue. The IO domain reached the 500-binary cap and stopped early, while FS and Mem ran until the 6-hour wall-clock budget was exhausted, explaining the slightly different totals across domains. The FS domain demonstrates the highest incompatibility rate at 86.8%, reflecting the complexity of VFS-layer semantics and the large number of flag combinations generated by the LLM-based mutations. The IO domain has the lowest rate (16.4%), attributable to the comparatively simpler argument space of I/O primitives. Memory management falls between the two at 63.6%, driven primarily by the intricacy of `mmap` flag validation and `madvise` advice semantics.

4.3 Filesystem Syscall Compatibility

Against Asterinas, 420 of 484 cases (86.8%) are incompatible. The detailed breakdown by incompatibility type is:

Incompatibility Type	Cases (FS)
`errno_diff`	386
`behavior_diff`	232
`ret_diff`	210
`unexpected_failure`	65
`unexpected_success`	35
`missing_syscall`	2

Note that a single incompatible case may exhibit multiple types of divergences simultaneously; consequently, the per-type counts sum to more than 420. `errno_diff` is the dominant signal (386 cases, 91.9% of all incompatible FS tests), indicating that Asterinas and Linux diverge most severely at the error-code level rather than simple success/failure outcome.

4.3.1 Errno Mapping Divergences

The single largest root cause is incorrect errno selection in error paths. When `fcntl` is called on an invalid file descriptor, Linux validates the descriptor first and returns `EBADF` (9); Asterinas evaluates the command code first and returns `EINVAL` (22) when the command is unsupported. A similar validation order inversion is observed for `openat` with invalid `dirfd` values: Linux returns `ENOTDIR` or `EINVAL` depending on the flag combination, while Asterinas returns `EEXIST` or `ENOENT` inconsistently. This pattern recurs across `fallocate`, `ftruncate`, and `fdatasync`, where Asterinas's internal error codes are mapped to a generic `EINVAL` rather than being translated to the correct Linux errno value.

4.3.2 Missing `openat2` Support

Asterinas return `ENOSYS` (38) for `openat2` (syscall 437), while Linux returns `EINVAL` (22) for malformed arguments. This indicates that `openat2`, introduced in Linux 5.6, is not yet implemented in the Rust-based kernel.

4.3.3 File-Descriptor Lifecycle Differences

Asterinas employs different file-descriptor validation logic. When writing to a `fd=0` (stdin) with a zero-length buffer succeeds on Linux, Asterinas returns `EBADF` (9). Some double close sequences which Linux handles through returing `EBADF`, Asterinas instead succeeds silently. `dup2` with an invalid source descriptor returns `ENOENT` (2) rather than `EBADF` (9), consistent with

an incorrect internal error translation in the fd-table lookup path. These fd-lifecycle differences will propagate to downstream syscalls and cause further incompatibilities in other areas which rely on the return value or the errno code.

4.4 I/O Syscall Compatibility

Against Asterinas, 82 of 500 cases (16.4%) are incompatible. The type breakdown is:

Incompatibility Type	Cases (IO)
`behavior_diff`	79
`ret_diff`	50
`errno_diff`	46
`unexpected_failure`	36
`unexpected_success`	15
`missing_syscall`	3

In contrast to the FS domain, `behavior_diff` (79 cases) is the leading signal in IO rather than `errno_diff`, indicating that the two platforms produce different observable side-effects even in cases where the error code matches.

4.4.1 `dup2` Returns `ENOENT` Instead of `EBADF`

The most reproducible IO incompatibility is `dup2` called with an invalid `oldfd`. Linux returns `EBADF` (9); Asterinas returns `ENOENT` (2). This is consistent with an incorrect error-code mapping in Asterinas's `dup2` syscall handler: the fd-table lookup path returns an internal "not-found" status that is translated to `ENOENT` rather than to the POSIX-mandated `EBADF`. The same bug affects `dup3` and `fcntl(F_SETOWN)` on invalid descriptors (returns `ESRCH` instead of `EBADF`).

4.4.2 `fcntl(F_GETFL)` Missing `O_LARGEFILE` Bit

`fcntl(F_GETFL)` in Asterinas returns a flag mask that is missing the `O_LARGEFILE` bit (`0x8000` on x86-64). Linux's open path sets this bit automatically for all 64-bit file opens; Asterinas either does not set it internally or truncates the flags word during kernel-to-userspace translation. Programs that inspect `O_LARGEFILE` to determine large-file support will incorrectly conclude the file was opened in 32-bit mode.

4.4.3 Access-Mode Check Divergence

`ftruncate` returns `EPERM` (1) in Asterinas for some files that Linux successfully truncates. Concurrently, `pwrite64` on a file descriptor opened `O_RDONLY` sometimes succeeds in Asterinas where Linux correctly returns `EBADF`. While the two divergences manifest in opposite ways, they can be traced back to a common cause: inconsistent permission checks in Asterinas's VFS compared to Linux.

4.4.4 Umask and File-Creation Mode Difference

Files created via `open(O_CREAT)` in Asterinas consistently show `st_mode = 0644`, while the same sequence on Linux produces `0664`. This indicates Asterinas applies `umask 0022` whereas the Linux test environment used `umask 0002`. Additionally, `st_uid`/`st_gid` in Asterinas are always 0, confirming that user-namespace credential mapping is not yet implemented.

4.4.5 `fdatasync` Stub Behavior

`fdatasync` on uninitialized or non-syncable file descriptors returns `0` (success) in Asterinas, while Linux returns `-1` with `errno = EINVAL`. This indicates that Asterinas's `fdatasync` handler lacks the file-descriptor type check performed by Linux's VFS layer before issuing the underlying sync operation.

4.5 Memory Management Syscall Compatibility

Against Asterinas, 304 of 478 cases (63.6%) are incompatible. The type breakdown is:

Incompatibility Type	Cases (Mem)
`errno_diff`	277
`behavior_diff`	251
`ret_diff`	110
`unexpected_success`	64
`unexpected_failure`	44
`missing_syscall`	1
`unknown`	3

`errno_diff` and `behavior_diff` are strongly co-occurring: 224 of 304 incompatible cases exhibit both simultaneously, suggesting a systematic pattern in which an incorrect errno is set alongside a divergent observable kernel-side state.

4.5.1 `mmap` Validation Order (68 Cases)

File-backed `mmap` with an invalid file descriptor should return `EBADF` (9) on Linux. Asterinas returns `EINVAL` (22) in 68 such cases, indicating that Asterinas validates `MAP_SHARED`/`MAP_PRIVATE` flag consistency and offset alignment **before** checking whether the descriptor is valid. Linux performs fd validity first, so `EBADF` takes precedence. This error-precedence inversion also manifests for `mprotect` and `mremap` with illegal address-range arguments (`ENOMEM` vs `EINVAL`).

4.5.2 `madvise` Incomplete Implementation

Three `madvise` advice values: `MADV_PAGEOUT`, `MADV_POPULATE_WRITE`, and `MADV_DONTNEED_LOCKED` are either unimplemented or incorrectly handled in Asterinas. Linux returns `ENOMEM` when the requested advice cannot be satisfied due to resource constraints; Asterinas returns `EINVAL`, representing a resource-exhaustion failure with an invalid-argument failure.

4.5.3 Struct ABI and `copy_to_user` Mismatches

In Asterinas, `writev` returns 73,312 bytes for an `iovec` sequence where Linux returns 4,096. This suggests a possible mismatch in how `iovec` lengths are interpreted or accumulated in the copy-from-user path. `getrusage` reports zeroed `ru_maxrss` and shifted `ru_stime` fields, which is consistent with a potential struct layout or ABI mismatch between the kernel and the userspace libc headers. In addition, failed `fstat` calls in Asterinas may leave parts of the `struct stat` buffer uninitialized instead of fully zeroing it before returning.

4.6 SAC Coverage Analysis

Equation (3.5.1.4) and Equation (3.5.1.6) in Section 2 define how ABIscope accumulates global coverage G_t across accepted seeds. Each experiment begins with a bootstrap phase in which all 200 initial seeds are ingested and their feature sets are merged into G_0. Subsequent LLM-based mutation iterations then expand the coverage set. The table below compares $|G_t|$ at the end of the bootstrap phase against its value at experiment termination. This quantifies the contribution of the mutation loop.

Domain	After Bootstrap (200 seeds)	After Mutations	Growth	Mutation Iters
Filesystem (FS)	1,502	4,383	×2.9	302
I/O Management (IO)	5,131	11,161	×2.2	444
Memory Mgmt (Mem)	1,204	3,226	×2.7	278

Even before any mutation, the 200 Syzkaller-derived seeds cover a non-trivial slice of each domain. FS bootstraps at 1,502 distinct feature tokens, a mix of flag co-occurrences, path-string patterns, and the usual numeric boundary constants. IO starts much higher, at 5,131, which is unsurprising given how many arguments syscalls like `dup2`, `ftruncate`, and `fcntl(F_GETFL)` each contribute on their own. Mem reaches at 1,204, close to FS; the flag spaces of `mmap`, `mprotect`, and `madvise` are wide but the seeds only sample a handful of typical combinations.

The mutation loop more than doubles coverage in all three domains. In FS, coverage goes from 1,502 to 4,383 over 302 iterations (2.9×); the five mutation strategies clearly find flag and path combinations that the initial seeds miss. IO ends at 11,161 after 444 iterations (2.2×). The per-accepted-seed yield is about 136 new tokens, roughly 13× higher than FS which we attribute to IO mutations hitting large-offset arithmetic and flag-mask edge cases that the seed corpus barely covers. Mem grows from 1,204 to 3,226 across 278 mutation files (2.7×), suggesting that even simple argument-level mutations on `mmap`, `madvise`, and `mprotect` keep producing novel runtime behavior. Overall, uniform random selection over the five mutation types is enough to keep SAC growing for the full 6-hour budget.

5. Summary and Future Work

5.1 Summary

This thesis presents ABIscope, a feedback-driven framework for automatically generating and evaluating user-space test cases that expose ABI incompatibilities between Linux and Rust-based kernels. The core pipeline integrates LLM-based seed translation and mutation, coverage-guided selection via the SAC metric, differential execution, and LLM-assisted attribution into a closed feedback loop that requires no kernel instrumentation.

5.1.1 Contributions and Research Significance

To the best of our knowledge, ABIscope is the first framework to apply LLM-guided mutation for systematic discovery of ABI incompatibilities between Linux and Rust-based kernels. Prior work on kernel fuzzing (e.g., Syzkaller) targets crash-inducing bugs and is not designed to evaluate the correctness of return values, errno codes, or side-effects; existing ABI analysis tools (e.g., SCML) perform only static flag-matching and do not execute programs to observe dynamic behavior. ABIscope fills this gap.

Its two core contributions are: (1) SAC (System-call Argument Coverage), a semantics-oriented feedback metric that quantifies argument-space diversity without kernel instrumentation, providing a principled selection signal suited to ABI compatibility analysis; and (2) a closed-loop pipeline that combines LLM-based mutation with differential execution and automated attribution, demonstrating that ABI incompatibilities in a real-world Rust-based kernel can be discovered at scale without manual effort.

The broader research significance lies in showing that the gap between a kernel claiming Linux ABI compatibility and one that actually achieves it is both measurable and substantial. ABIscope provides a practical, reusable methodology for kernel developers to continuously verify and narrow this gap as their implementations mature.

5.1.2 Limitations

Despite these contributions, three limitations should be acknowledged.

First, all experiments target a single Rust-based kernel, Asterinas. While Asterinas is a representative and actively maintained Linux ABI-compatible kernel, conclusions may not generalise to other Rust-based kernels such as Kerla or rCore. The results may differ because of different syscall implementation and maturity levels.

Second, the evaluation covers only three syscall subsystems: filesystem operations, I/O management, and memory management. Important subsystems including networking (`socket`/`bind`/`sendmsg`), inter-process communication (pipes, signals, shared memory), and device-driver interfaces remain untested. ABI gaps in those areas may be equally or more severe.

Third, all generated test programs are single-threaded and stateless across executions. Concurrency-related ABI issues such as race conditions in fd allocation, atomic behavior differences in `futex`, or scheduler-dependent errno sequencing cannot be captured by the current framework, since the differential execution model assumes deterministic output for the same input[32].

5.2 Future Work

The limitations above suggest several directions for future work.

The most immediate direction is to expand the kernel set. Adding Kerla and rCore as additional targets would help determine whether the incompatibilities we observe are specific to Asterinas or reflect broader patterns in Rust-based reimplementations of the Linux ABI. We expect that some of the IO-related findings may carry over, while the file system behavior differences may be less consistent across implementations.

A second direction is to extend coverage beyond the three domains studied in this work. Networking and IPC syscalls can, in principle, be supported within the same SAC and mutation pipeline. However, they require additional flag dictionaries and new templates for socket addresses and message buffers. While this extension does not appear conceptually difficult, it is likely to involve substantial engineering effort.

Concurrency presents a more challenging and interesting direction. The current framework executes each test in a single-threaded setting, which means that issues dependent on interleavings, such as lock ordering problems, missed memory barrier effects, or inconsistent updates to file descriptor tables, are not captured. Extending differential comparison to multi-threaded execution traces would require defining equivalence between executions where ordering is no longer fixed, which remains an open problem.

Finally, the current system uses the LLM only for input generation and mutation. It may also be possible to use the model to suggest fixes for confirmed incompatibilities, given divergence traces and relevant source context. However, it remains unclear whether such suggestions would be practically useful, or whether they would simply produce plausible but

superficial patches that pass the existing test suite. This would need careful evaluation before any stronger claims could be made.

References

[1] WANG Y J. A Summary of Research on the Linux Operating System[J/OL]. Advanced Materials Research, 2013, 705: 565-568. https://api.semanticscholar.org/CorpusID:62728732.

[2] SZEKERES L, PAYER M, WEI T, et al. SoK: Eternal War in Memory[C/OL]//Proceedings of the 2013 IEEE Symposium on Security and Privacy. USA: IEEE Computer Society, 2013: 48-62. https://doi.org/10.1109/SP.2013.13.

[3] KELLERMANN M. The Dirty Pipe Vulnerability (CVE-2022-0847)[M]. 2022.

[4] PENG Y, TIAN H, ZHANG J, et al. ASTERINAS: a Linux ABI-compatible, rust-based framekernel OS with a small and sound TCB[C]//Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference. Boston, MA, USA: USENIX Association, 2025.

[5] CHEN S F, WU Y S. Linux Kernel Module Development with Rust[C]//2022 IEEE Conference on Dependable and Secure Computing (DSC). 2022: 1-2.

[6] SOCHAT V, HAINES T. Binary-level Software Compatibility Tool Agreement[EB/OL]. (2023). https://arxiv.org/abs/2212.03364.

[7] BOOS K, LIYANAGE N, IJAZ R, et al. Theseus: an Experiment in Operating System Structure and State Management[C/OL]//14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). USENIX Association, 2020: 1-19. https://www.usenix.org/conference/osdi20/presentation/boos.

[8] LAN Y, JIN D, WANG Z, et al. Thunderkaller: Profiling and Improving the Performance of Syzkaller[C/OL]//Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering. Echternach, Luxembourg: IEEE Press, 2024: 1567-1578. https://doi.org/10.1109/ASE56229.2023.00124.

[9] JONES D. Trinity: A Linux System Call Fuzzer[M]. 2010.

[10] LINUX TEST PROJECT. Linux Test Project (LTP)[M].

[11] CHEN M, TWOREK J, JUN H, et al. Evaluating Large Language Models Trained on Code[EB/OL]. (2021). https://arxiv.org/abs/2107.03374.

[12] LU S, GUO D, REN S, et al. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation[EB/OL]. (2021). https://arxiv.org/abs/2102.04664.

[13] CHEN H, MAO Y, WANG X, et al. Linux kernel vulnerabilities: state-of-the-art defenses and open problems[C/OL]//Proceedings of the Second Asia-Pacific Workshop on Systems. Shanghai, China: Association for Computing Machinery, 2011. https://doi.org/10.1145/2103799.2103805.

[14] LOVE R. Linux kernel development[M]. Pearson Education, 2010.

[15] RESHETOVA E, LILJESTRAND H, PAVERD A, et al. Toward Linux kernel memory safety[J/OL]. Software: Practice and Experience, 2018, 48(12). http://dx.doi.org/10.1002/spe.2638.

[16] BUGDEN W, ALAHMAR A. Rust: The Programming Language for Safety and Performance[EB/OL]. (2022). https://arxiv.org/abs/2206.05503.

[17] KAN S, CHEN Z, SANAN D, et al. An Executable Operational Semantics for Rust with the Formalization of Ownership and Borrowing[EB/OL]. (2020). https://arxiv.org/abs/1804.07608.

[18] RCORE-OS. rCore[EB/OL]. (2026). https://github.com/rcore-os/rCore.

[19] NUTA. Kerla[EB/OL]. (2026). https://github.com/nuta/kerla.

[20] BOEHME M, CADAR C, ROYCHOUDHURY A. Fuzzing: Challenges and Reflections[J]. IEEE Software, 2021, 38(3): 79-86.

[21] LYU Y, XIE Y, CHEN P, et al. Prompt Fuzzing for Fuzz Driver Generation[C/OL]//Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. Salt Lake City, UT, USA: Association for Computing Machinery, 2024: 3793-3807. https://doi.org/10.1145/3658644.3670396.

[22] LIU Y, DENG J, JIA X, et al. PromeFuzz: A Knowledge-Driven Approach to Fuzzing Harness Generation with Large Language Models[C/OL]//Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. Taipei, Taiwan: Association for Computing Machinery, 2025: 1559-1573. https://doi.org/10.1145/3719027.3765222.

[23] GODEFROID P, PELEG H, SINGH R. Learn&Fuzz: machine learning for input fuzzing[C]//Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering. Urbana-Champaign, IL, USA: IEEE Press, 2017: 50-59.

[24] YUAN Z, LIU M, DING S, et al. Evaluating and Improving ChatGPT for Unit Test Generation[J/OL]. Proc. ACM Softw. Eng., 2024, 1(FSE). https://doi.org/10.1145/3660783.

[25] SOLHEIM J, ROWLAND J. An empirical study of testing and integration strategies using artificial software systems[J]. IEEE Transactions on Software Engineering, 1993, 19(10): 941-949.

[26] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Proceedings of the 31st International Conference on Neural Information Processing Systems. Long Beach, California, USA: Curran Associates Inc., 2017: 6000-6010.

[27] RADFORD A, NARASIMHAN K, SALIMANS T, et al. Improving language understanding by generative pre-training[J]. 2018.

[28] CHEN M, TWOREK J, JUN H, et al. Evaluating Large Language Models Trained on Code.[J/OL]. CoRR, 2021. http://dblp.uni-trier.de/db/journals/corr/corr2107.html#abs-2107-03374.

[29] NIE Y, LI H, GUO C, et al. VulnLLM-R: Specialized Reasoning LLM with Agent Scaffold for Vulnerability Detection[EB/OL]. (2025). https://arxiv.org/abs/2512.07533.

[30] LEFEUVRE H, GAIN G, BĂDOIU V A, et al. Loupe: Driving the Development of OS Compatibility Layers[C/OL]//Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1. La Jolla, CA, USA: Association for Computing Machinery, 2024: 249-267. https://doi.org/10.1145/3617232.3624861.

[31] KLEES G, RUEF A, COOPER B, et al. Evaluating Fuzz Testing[C/OL]//Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. Toronto, Canada: Association for Computing Machinery, 2018: 2123-2138. https://doi.org/10.1145/3243734.3243804.

[32] ETEMADI K, MOHAMMADI B, SU Z, et al. Mokav: Execution-driven differential testing with LLMs[J/OL]. J. Syst. Softw., 2025, 230(C). https://doi.org/10.1016/j.jss.2025.112571.